Public transport ticket pricing backend [redacted]

You are a software engineer, provide a backend function for this system in python:
The system is a app for public transport system where each passenger needs are met, tickets can be bought for a single ride, a set of period or in bundles. Passengers should be able to buy tickets through mobile application or other digital interface. System should determine how much each passenger should pay depending on who they are (student, teenagers, seniors, normal adult) and the type of ticket. system should handle passengers purches multiple tickets at once. The system should handle unexpected or inusual inputs such as selecting a ticket without specifying a valid quantity. The system should handle high amount of request in short period of time and protect personal data.

Check frontend design skill for any relevant patterns >

Here's a production-grade backend module. I'll structure it across clean, well-separated concerns:

Created 7 files, ran a command >

37/37 tests passing. Let me present the files:

Presented 5 files >

Architecture walkthrough

The system is split into **4 focused modules**, each with a single responsibility. Here's why each decision was made:

Module structure

File	Responsibility	Why isolated
`models.py`	Domain types, pricing tables	Single source of truth for pricing — change rates here, nowhere else

File	Responsibility	Why isolated
`validation.py`	Input sanitisation	Collects **all** errors before returning, avoiding waterfall UX
`pricing.py`	Pure pricing logic	Zero I/O → trivially unit-testable, no mocks needed
`api.py`	HTTP, rate limiting, idempotency	All infrastructure concerns stay out of business logic

---

Key design decisions and why

`Decimal` for money, never `float` `float` arithmetic introduces rounding drift — `0.1 + 0.2 != 0.3` in Python. For anything that flows into a payment processor, you use `Decimal` with explicit `ROUND_HALF_UP`. Non-negotiable.

Validation collects all errors before responding Rather than raising on the first bad field, `validate_purchase_request` walks the entire payload and returns the full list. A mobile user who forgets both `quantity` and `ticket_type` gets told both things at once — not in two roundtrips.

Quote-then-confirm pattern (not purchase-in-one-shot) The endpoint returns a `PurchaseQuote`, not a ticket. Ticket issuance should only happen in a `/confirm` endpoint after the payment processor confirms funds. This prevents the common race condition where a ticket is issued but the charge fails.

Idempotency via Redis Mobile clients on flaky networks retry. Without idempotency, a retry creates a duplicate charge. The `X-Idempotency-Key` header + 24h Redis cache returns the same response for repeat requests. Key is hashed before being logged — never stored raw.

Rate limiting per passenger ID, not IP IP-based limiting breaks behind NATs and mobile carriers (shared IPs). Keyed on the passenger ID (hashed in logs). Config is env-var driven so you can tune it per deployment without a code change.

PII discipline `passenger_id` is a backend-assigned opaque reference, never a name or email. In every log line it's one-way hashed with SHA-256. The `PurchaseQuote` carries only the anonymised ID, never contact details.

What's intentionally left out (and what to add next)

- **Authentication** — The endpoint assumes a valid JWT is already verified by an upstream gateway (Kong, AWS API Gateway, etc.). Add `Depends(verify_jwt)`

when wiring into your auth stack.

- **Database persistence** — Quotes and confirmed tickets need to be written to Postgres (or equivalent) inside a transaction. The `build_purchase_quote` result is the DTO to persist.
- **Payment integration** — Wire Stripe/Adyen in the `/confirm` endpoint; pass `quote.request_id` as the payment metadata for reconciliation.
- **Distributed rate limiting** — The current Redis `INCR` approach is correct for a single region. For multi-region, use Redis Cluster or a Lua script for atomic sliding-window counters.

I want a very brief code and prefer to have everything in one class

Created a file, ran a command, read a file

~100 lines, one class. Here's what it does:

`TicketingService` has one public method — `purchase(payload)` — and three private ones:

- `_validate` — collects **all** errors across the whole payload before raising, so the client gets everything wrong in one shot
- `_validate_item` — per-line-item checks: missing ticket type, missing/zero/non-integer quantity, exceeding max
- `_price_items` — applies the category discount multiplier and computes line totals using `Decimal` (float would introduce rounding drift in monetary values)

The `Enum` classes and `dataclass` results live outside the class intentionally — they're pure data types, not behaviour, so putting them inside would just add noise.

To wire this into FastAPI or Django you'd call `svc.purchase(request.json())` and catch `ValueError` to return a 422 response.